WEEK 3 · LESSON 15 OF 24 · HANDOUT

Retrieval-Augmented Generation (RAG)

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=vSwK5sBpD2c — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython 2024: Is RAG all you need? A look at the limits of retrieval augmented generation — https://www.youtube.com/watch?v=9wk7mGB_Gp4

🎓 **Go deeper:** PyData Berlin 2023: How to Feed Facts to Large Language Models and Reduce Hallucination — https://www.youtube.com/watch?v=-eRjgj1zww4

Your goal in this lesson: Build a complete RAG pipeline that retrieves the right sections, asks the model to answer only from them with citations, and says "I don't know" when the answer is not there.

Why RAG?

An LLM only knows what was in its training data — not your organisation's handbook, today's prices or last week's decisions — and it may **invent** an answer rather than admit it does not know. **Retrieval-Augmented Generation** fixes this:

1. **Retrieve** the most relevant passages from your documents.
2. **Augment** the prompt with them.
3. **Generate** an answer grounded in those passages, with citations.

Updating knowledge becomes easy: change the documents, not the model.

```
import importlib.util, os, subprocess, sys

if importlib.util.find_spec("anthropic") is None:          # install once per Colab session
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "anthropic"], check=True)

try:
    from google.colab import userdata                     # Colab: key saved in the Secrets
panel (key icon)
    os.environ["ANTHROPIC_API_KEY"] = userdata.get("ANTHROPIC_API_KEY")
except ImportError:
    pass                                                    # not in Colab: set
ANTHROPIC_API_KEY in your environment
except Exception:
    print("Add a secret called ANTHROPIC_API_KEY in Colab's Secrets panel, then run this cell
again.")

import anthropic

client = anthropic.Anthropic()        # reads ANTHROPIC_API_KEY from the environment
MODEL = "claude-haiku-5-5"            # fast and low-cost; check docs.claude.com for current
model names

def ask(prompt, system=None, max_tokens=500):
    extra = {"system": system} if system else {}
    response = client.messages.create(model=MODEL, max_tokens=max_tokens,
                                      messages=[{"role": "user", "content": prompt}], **extra)
    return response.content[0].text
```

```
HANDBOOK = {
    "Opening hours": "The Kigali Tech Hub is open Monday to Friday from 8:00 to 20:00 and on
Saturday "
                     "from 9:00 to 14:00. The hub is closed on Sundays and on public
holidays.",
    "Membership plans": "There are three plans. The Student plan costs 10,000 RWF per month and
needs a "
                        "valid student card. The Standard plan costs 25,000 RWF per month. The
Team plan "
                        "costs 90,000 RWF per month for up to five people.",
    "Joining": "To become a member, fill in the online form, bring your national ID or passport
to the "
               "front desk, and pay the first month. Your access card is ready the same day.",
    "Payment": "Fees are paid by mobile money or bank transfer before the 5th of each month.
Members who "
               "pay late for two months lose access until the balance is cleared.",
    "Cancelling": "You may cancel your membership at any time with 14 days' written notice by
email. "
                  "Fees already paid for the current month are not refunded.",
    "Wi-Fi": "The Wi-Fi network is called KTH-Members. The password is printed on your access
card and "
             "changes every three months. Guests use the separate KTH-Guest network.",
    "Meeting rooms": "Members can book the two meeting rooms through the hub app. Standard and
Team members "
                     "get 4 free hours per month; Student members get 1 free hour. Extra hours
cost 5,000 RWF.",
    "Guests": "Each member may bring up to two guests per day. Guests must sign in at the front
desk "
              "and leave by 18:00.",
    "Printing": "Black-and-white printing costs 50 RWF per page and colour printing 200 RWF per
page. "
                "Students get 100 free black-and-white pages per month.",
    "Events": "The hub runs a free coding workshop every Wednesday at 17:00 and a startup pitch
night "
              "on the last Friday of each month. Members can register in the hub app.",
    "Lockers": "Lockers can be rented for 3,000 RWF per month. Lost locker keys cost 5,000 RWF
to replace.",
    "Code of conduct": "Members must respect others, keep noise low in the quiet zone, and
never share their "
                       "access card. Harassment leads to immediate cancellation of
membership.",
    "Lost and found": "Items left behind are kept at the front desk for 30 days, then
donated.",
    "Accessibility": "The ground floor, meeting rooms and toilets are wheelchair accessible. A
lift serves "
                     "all floors. Contact the front desk to arrange other support.",
}
chunks = [{"id": i, "title": t, "text": f"{t}. {body}"} for i, (t, body) in
enumerate(HANDBOOK.items())]
print(len(chunks), "sections")
```

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import Normalizer

texts = [c["text"] for c in chunks]
embedder = make_pipeline(TfidfVectorizer(stop_words="english", sublinear_tf=True),
                         TruncatedSVD(n_components=12, random_state=0),
                         Normalizer())                      # unit length: dot product = cosine
similarity
doc_vectors = embedder.fit_transform(texts)

def embed(text_list):
    return embedder.transform(text_list)

print("each section is now a vector of", doc_vectors.shape[1], "numbers")
```

Step 1 — Retrieve

```
def retrieve(question, k=3):
    scores = doc_vectors @ embed([question])[0]
    best = np.argsort(scores)[::-1][:k]
    return [chunks[i] for i in best]

for c in retrieve("How much is the student plan?"):
    print("-", c["title"])
```

Step 2 — Build a grounded prompt

```
RAG_SYSTEM = (
    "You answer questions about the Kigali Tech Hub using ONLY the numbered sources provided. "
    "Cite sources like [1] after each fact. If the sources do not contain the answer, reply
exactly: "
    "\"I don't know based on the handbook.\" Never use outside knowledge."
)

def build_prompt(question, sources):
    numbered = "\n".join(f"[{n}] {s['text']}" for n, s in enumerate(sources, start=1))
    return f"<sources>\n{numbered}\n</sources>\n\n<question>{question}</question>"

sources = retrieve("How much is the student plan?")
print(build_prompt("How much is the student plan?", sources))
```

The prompt contains the question **and** the evidence. The system prompt sets strict rules: only the sources, cite them, admit when the answer is missing.

Step 3 — Generate, with citations you can check

```
import re

def answer(question, k=3):
    sources = retrieve(question, k)
    text = ask(build_prompt(question, sources), system=RAG_SYSTEM, max_tokens=300)
    cited = sorted({int(n) for n in re.findall(r"\[(\d+)\]", text) if 1 <= int(n) <=
len(sources)})
    return {"answer": text, "sources": [sources[n - 1]["title"] for n in cited],
            "retrieved": [s["title"] for s in sources]}

result = answer("How much is the student plan?")
print(result["answer"])
print("cited:", result["sources"], "| retrieved:", result["retrieved"])
```

Because citations are numbers, your app can **show the cited sections** next to the answer, so people can check it. Answers with no valid citation can be flagged.

Step 4 — Questions the handbook cannot answer

```
print(answer("Who won the football match yesterday?")["answer"])
```

A good RAG system answers "I don't know based on the handbook." A system that answers anyway is hallucinating — test for this on purpose.

Where RAG goes wrong

Problem	Symptom	Fix (next lessons)
Wrong passages retrieved	Answer misses the point or says "I don't know" wrongly	Better chunking, hybrid search, reranking
Right passages, wrong answer	Misreads numbers, mixes plans	Clearer prompt, smaller context, better model
Answer goes beyond sources	Invented details	Strict instructions, citation checks, evaluation
Outdated documents	Correct citation, old fact	Keep the document store up to date

The first one is the most common: **if retrieval fails, generation cannot succeed.** That is why the next lesson focuses on retrieval.

Key terms

- **RAG** — retrieve relevant passages, add them to the prompt, generate a grounded answer.
- **Grounding** — basing the answer on supplied evidence.
- **Citation** — a pointer from a claim to its source.
- **Context stuffing** — putting retrieved passages in the prompt.
- **Refusal ("I don't know")** — the correct behaviour when the sources lack the answer.

Remember this: RAG gives an LLM your organisation's knowledge at question time; ground every answer in retrieved sources, cite them, and make "I don't know" the expected reply when the answer is missing.

Name: _____ Date: _____

Activity

Task 1

Ask five questions from the handbook and check every citation against the source text.

Task 2

Change `k` to 1 and to 6. When does a small k miss information? When does a large k confuse the model?

Task 3

Remove "Never use outside knowledge" from the system prompt and ask about the football match again.

Task 4

Add a new section "Parking" to the handbook, rebuild the embeddings, and ask about parking.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What problem does RAG solve?
   - A. Tokenization
   - B. The model not knowing your own, current documents and inventing answers
   - C. Slow GPUs
   - D. Image generation
2. What are the three steps of RAG?
   - A. Ask, wait, print
   - B. Tokenize, embed, sort
   - C. Train, test, deploy
   - D. Retrieve, augment, generate
3. Why number the sources and ask for citations like [1]?
   - A. To make the prompt longer
   - B. Numbers are cheaper
   - C. The API requires it
   - D. So people and code can check each claim against its source
4. What should a good RAG system answer when the sources lack the answer?
   - A. Its best guess
   - B. "I don't know based on the handbook."
   - C. A random source
   - D. Nothing
5. What is the most common cause of bad RAG answers?
   - A. Short questions
   - B. Too many citations
   - C. Retrieval returning the wrong passages
   - D. The font
6. How do you update a RAG system's knowledge?
   - A. Retrain the model
   - B. Update the documents and their vectors
   - C. You cannot
   - D. Change the API key

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.